

V1 SOFTWARE AND TOOLCHAIN BASELINE

Autonomous Semantic Navigation Robot

Jan Cuesta

Isaac Sim 6.1.0 + ROS 2 Jazzy

Version 0.9 – Development baseline – 24 September 2026

Purpose. Freeze the working V1 software combination, define a lightweight update policy, and provide an automatic version-capture method sufficient for reproducible implementation and acceptance evidence.

This baseline avoids package-by-package bureaucracy: stable platform choices are locked, while exact installed package

revisions and Git commits are captured automatically at milestones.

DOCUMENT CONTROL

Small baseline, automatic evidence

Field	Value
Document owner	Jan Cuesta
Project	Autonomous Semantic Navigation Robot
Document identifier	ASN-V1-SW-BASE
Revision / status	0.9 / implementation baseline
Related records	V1 ConOps v0.9; V1 Requirements and Risk v0.9; Physical Interface v0.9
Companion tool	<code>capture_software_baseline.sh</code>
Tailoring principle	Lock compatibility boundaries; automate detailed inventory; do not delay coding for cosmetic version records.

Revision and approval

Version	Date	Change	Status
0.9	24 Sep 2026	Initial working platform, package policy, capture procedure, and update gate.	Use for V1
1.0	Before acceptance	Attach final manifest and replace remaining captured-at-run fields.	Release baseline

Step 4 result. Continue with the installed Isaac Sim 6.1.0 / Ubuntu 24.04.5 / ROS 2 Jazzy combination. Do not upgrade the simulator, ROS distribution, or middleware during V1 unless a blocking defect requires it.

1. Status meanings

Status	Meaning
Locked	Compatibility boundary for V1. Change only through the update gate.
Observed	Previously read from the installed computer; capture again for formal evidence.
Repository-controlled	Version is the project Git commit or a dependency commit/tag.
Captured at run	Exact revision is written by the companion script for each milestone.
V2 target	Physical-robot value; verify on the ASU unit rather than forcing it into V1.

01 / LOCKED V1 STACK

The combination we will build on

2. Platform baseline

Layer	Baseline	Status	Control
Operating system	Ubuntu 24.04.5 LTS, x86_64	Locked / observed	Native dual-boot installation
Kernel	7.0.0-31-generic observed	Captured at run	Not a strict compatibility pin
Simulator	NVIDIA Isaac Sim 6.1.0	Locked / observed	Standalone Linux x86_64 ZIP installation
ROS distribution	ROS 2 Jazzy Jalisco	Locked	Ubuntu deb packages in <code>/opt/ros/jazzy</code>
ROS middleware	Fast DDS, <code>rmw_fastrtps_cpp</code>	Locked for V1	Set explicitly in controlled launcher
Navigation	Nav2 Jazzy release track; 1.3.1 reference	Locked API track	Installed binary packages; exact deb revision captured
Mapping	SLAM Toolbox Jazzy; 2.8.5 reference	Locked API track	Exact deb revision captured
Localization	Nav2 AMCL, Jazzy package set	Locked API track	Parameters in project YAML
Visualization	RViz 2 from ROS 2 Jazzy	Locked API track	Exact deb revision captured
Logging	rosbag2 from ROS 2 Jazzy	Locked API track	Storage plugin and revision captured
Language standard	C++17	Locked	Enforced in CMake
Python runtime	Python 3.12 system/Isaac-compatible runtime	Locked boundary	Launch and support tooling only
Build system	CMake + ament_cmake + colcon	Locked workflow	Exact tool revisions captured
Version control	Git; project commit recorded per run	Repository-controlled	Clean/dirty state recorded
GPU driver	NVIDIA 580.173.02 observed	Captured at run	Change only after V1 checkpoint
GPU	RTX 4060 Laptop GPU, 8.59 GB reported VRAM	Observed hardware	Performance profile remains conservative

CUDA interpretation. The ‘nvidia-smi’ CUDA 13.0 display is the highest CUDA version supported by the driver, not proof that a separate CUDA Toolkit 13.0 is installed. V1 does not require a standalone CUDA Toolkit unless a later component explicitly needs it.

3. Already installed and verified

The workstation previously installed or verified the Jazzy binary packages for Nav2, Nav2 bringup, SLAM Toolbox, point-cloud conversion, and the NVIDIA Isaac Sim ROS workspace. The exact deb versions and workspace commit are intentionally obtained by the capture script instead of transcribed manually.

4. Not part of the V1 lock

- Gazebo, ROS 1, Isaac ROS, CUDA development toolkit, Docker, and source-built Nav2 are not required for V1.
- The TurtleBot 4 package and Create 3 firmware revisions are V2 target facts and are captured on the ASU unit.
- RGB-D and perception package versions are deferred to V3.

02 / ENVIRONMENT CONTRACT

One launch environment, no silent mixing

5. Shell and ROS environment

Each controlled terminal shall source ROS 2 Jazzy before the project overlay. The launcher shall set the RMW implementation explicitly and shall not mix Humble, Jazzy, or multiple workspaces in the same shell.

Item	Rule
Base ROS environment	Source <code>/opt/ros/jazzy/setup.bash</code> .
Project overlay	Source the project <code>install/setup.bash</code> only after Jazzy.
Middleware	Export <code>RMW_IMPLEMENTATION=rmw_fastrtps_cpp</code> .
Simulation time	Set <code>use_sim_time=true</code> for every simulation-dependent node.
Isaac bridge	Enable the Isaac Sim ROS 2 extensions for Jazzy; do not load a Humble bridge.
Domain ID	Use one documented development value; capture it in every run.
Python packages	Use Ubuntu/ROS deps or a project-specific virtual environment; do not modify Isaac's bundled Python globally.

Shell hygiene. If a terminal has sourced the wrong distribution or an unrelated workspace, close it and open a clean terminal. Debugging a mixed environment costs more time than restarting it.

6. Dependency policy

- Prefer Ubuntu and ROS binary packages for Nav2, SLAM Toolbox, TF, RViz, and rosbag2.
- Build project-owned packages from source in the project workspace.
- Add a source dependency only when the binary release lacks required behavior. Record its repository URL, commit, license, and reason.
- Do not use floating ‘main’, ‘master’, nightly, or Rolling branches in acceptance evidence.
- Keep parameters and launch files in Git; do not edit files under `/opt/ros/jazzy`.

7. Repository state rule

Every evidence-producing run records the project commit. Development runs may use a dirty tree, but the manifest must say so. Formal acceptance runs require either a clean tree or a committed, archived diff explicitly linked to the result.

8. Performance profile

The existing 30 FPS laptop-safe Isaac Sim profile remains the default development configuration. Performance settings are configuration, not a new software version. A higher-quality profile may be used for demonstrations only after the baseline path remains functional.

03 / AUTOMATIC VERSION CAPTURE

Run it; do not type inventories manually

9. Companion capture tool

The companion shell script records the operating system, kernel, GPU/driver, ROS environment, core development tools, relevant ROS deb packages, project Git state, optional Isaac Sim directory metadata, and optional Isaac Sim ROS workspace commit.

From the future repository root, run:

```
./tools/capture_software_baseline.sh
. /path/to/isaacsim [/path/to/IsaacSim-ros_workspaces]
```

Until the repository skeleton exists, the delivered script may be run from its current location with the intended project directory as the first argument. It creates a timestamped text manifest under `evidence/software/`; it does not install, update, or remove software.

10. When to capture

Moment	Reason
Repository initialization	Establish the implementation starting point.
First integrated map	Identify the toolchain that produced the approved map.
First successful navigation	Tie behavior to exact packages and commit.
Before/after approved upgrade	Make the compatibility change auditable.
Formal V1 acceptance	Freeze the release evidence environment.
First ASU robot session	Add V2 robot packages, firmware, RMW, and network facts.

11. Required manifest contents

A valid manifest contains timestamps, OS/kernel, GPU driver, ROS distribution and RMW, package revisions for Nav2/SLAM/TF/rosbag2, compiler/build-tool versions, project commit and dirty state, and any external workspace commit. Missing optional tools are reported as unavailable and do not fail capture.

12. Acceptance evidence naming

Use the generated UTC timestamp. Do not rename a manifest after associating it with a rosbag or trial result. The trial metadata references the manifest filename and project commit.

04 / UPDATE GATE

Stable enough to learn; flexible enough to unblock

13. Allowed without baseline revision

- Editing project source, launch, configuration, tests, worlds, maps, and documentation under Git control.
- Installing a missing Jazzy binary dependency that does not replace the locked simulator, ROS distribution, or middleware.
- Applying normal Ubuntu security updates, provided a new manifest and smoke test are completed before evidence runs.

14. Requires an update decision

Changing Isaac Sim, Ubuntu release, ROS distribution, RMW implementation, Nav2/SLAM API track, or compiler language standard requires:

1. record the reason and expected benefit;

2. capture the pre-change manifest;
3. create a Git branch or checkpoint;
4. update one layer at a time;
5. rebuild from a clean workspace and run the smoke tests;
6. capture the post-change manifest and either accept or revert the change.

15. Smoke-test gate

ID	Check	Evidence
SW-01	Clean terminal sources Jazzy and project overlay without errors.	Console log
SW-02	Project packages build with C++17 and tests execute.	colcon output
SW-03	Isaac Sim starts with the ROS 2 bridge and publishes ‘/clock’.	Topic evidence
SW-04	LiDAR, odometry, TF, and robot description are present and connected.	CLI + RViz
SW-05	SLAM Toolbox starts and produces a map from simulated sensing.	Map + log
SW-06	Saved-map localization reaches Ready without initialization motion.	Diagnostic record
SW-07	Nav2 accepts a goal, publishes feedback, commands motion, and cancels safely.	Bag + result
SW-08	Manual Safety Stop, simulated E-stop, and command watchdog stop motion.	Stop test
SW-09	Required evidence and a complete software manifest are preserved.	Inspection

Exit condition. Version 0.9 is sufficient to begin Step 5 architecture and Step 6 repository creation. Exact deb package strings and Git commits become authoritative when the first manifest is generated on Jan’s Ubuntu workstation.

16. References

1. NVIDIA Isaac Sim 6.1 release notes. https://docs.isaacsim.omniverse.nvidia.com/latest/overview/release_notes.html
2. NVIDIA Isaac Sim ROS 2 installation: Ubuntu 24.04 and ROS 2 Jazzy. https://docs.isaacsim.omniverse.nvidia.com/6.0.0/installation/install_ros.html
3. ROS 2 Jazzy Ubuntu 24.04 installation documentation. <https://docs.ros.org/en/jazzy/Installation/Alternatives/Ubuntu-Install-Binary.html>
4. Nav2 Jazzy binary installation and release workflow. https://docs.nav2.org/jazzy/getting_started/build_and_install/local_installation/
5. SLAM Toolbox Jazzy 2.8.5 documentation. https://docs.ros.org/en/jazzy/p/slam_toolbox/